Numerical Methods & Applied Programming (ENG2124)

NumPy for Arrays

Dr Ian (Yinglong) He

A: 14b AC 03

E: yh0015@surrey.ac.uk

T: 01483 688784

Content

- What is NumPy?
- NumPy array
- Array operations
- Linear algebra
- Fourier transform
- Random sampling

What is NumPy?

NumPy ("Numerical Python")

- NumPy is a Python package used for working with ***arrays*** and created in 2005 by Travis Oliphant.
- Python list can serve the purpose of arrays, but slow to process. NumPy ndarray (N-dimensional array object) is 50x faster.
- Fundamental package for scientific computing with Python, with capabilities in linear algebra, matrices, Fourier transform, random number, etc.
- Building block for other packages (e.g. Pandas, and SciPy)
- Source code: https://github.com/numpy/numpy.

NumPy array

Creating array

- We'll start with the standard NumPy import, under the alias np.

```
import numpy as np
```

- Creating arrays from Python list

```
# integer array:
np.array([[1, 2, 3], [4, 5, 6]])
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
# float array:
np.array([[1, 2, 3], [4, 5, 6]], float)
```

```
array([[1., 2., 3.],
       [4., 5., 6.]])
```

- NumPy is constrained to arrays that all contain the same type. If types do not match, NumPy will upcast if possible, e.g.,

```
np.array([3.14, 4, 2, 3])
```

```
array([3.14, 4. , 2. , 3. ])
```

NumPy array

- Creating arrays from scratch

```
# Create an array starting at 0, ending at 20, stepping by 2
np.arange(0, 20, 2)
```

```
array([ 0, 2, 4, 6, 8, 10, 12, 14, 16, 18])
```

```
# Create an array of five values evenly spaced between 0 and 1
np.linspace(0, 1, 5)
```

```
array([0. , 0.25, 0.5 , 0.75, 1. ])
```

- Arrays are mutable

```
# Create a 2x3 array filled with zeros
a = np.zeros((2,3))
print(a)
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
a[0, 0] = 1
print(a)
```

```
array([[1., 0., 0.],
       [0., 0., 0.]])
```

NumPy array

File input/output

- Save/read an array to/from a text file.

```
# Create a 3x3 array of uniformly distributed random values between 0 and 1
a = np.random.random((3, 3))
print(a)
```

```
array([[0.94939114, 0.19356698, 0.16297283],
       [0.84066918, 0.63034489, 0.45956742],
       [0.63581301, 0.76601824, 0.83719879]])
```

```
# save to file
np.savetxt("a_out.txt", a)

# read from file
b = np.loadtxt("a_out.txt")
print(b)
```

```
array([[0.94939114, 0.19356698, 0.16297283],
       [0.84066918, 0.63034489, 0.45956742],
       [0.63581301, 0.76601824, 0.83719879]])
```

NumPy array

Attributes of arrays

- Determine the size, shape, memory consumption, and data types of arrays.

```
a = np.random.random((2, 5))
print(a)
```

```
array([[0.88793416, 0.95292026, 0.30999826, 0.61001556, 0.64699089],
       [0.24566477, 0.65684341, 0.22757806, 0.73622901, 0.85446859]])
```

```
a.ndim  # 2 dimension
a.shape # (2, 5) shape of array
a.size  # 10 # of elements
a.T     # transpose
a.dtype # data type
```

NumPy array

Reshaping arrays

- Reshape using reshape method. Total size must remain the same.

```
a = np.arange(1, 10)
print(a)
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
b = a.reshape((3, 3))
print(b)
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

NumPy array

Slicing arrays

- We can use square brackets to access subarrays with the slice notation, marked by the colon (:) character.

```
a = np.arange(12).reshape(3, 4)
print(a)
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
# third row, all columns
a[2, :]
```

```
array([ 8, 9, 10, 11])
```

```
# 2nd, 3rd row, all columns
a[1:3]
```

```
array([[ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
# all rows, columns 3 and 4
a[:, 2:4]
```

```
array([[ 2,  3],
       [ 6,  7],
       [10, 11]])
```

Joining arrays

- Joining of two arrays in NumPy, is primarily accomplished using the methods concatenate, vstack, and hstack.

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a)
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
# concatenate along the 1st axis
np.concatenate([a, a])
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [1, 2, 3],
       [4, 5, 6]])
```

```
# concatenate along the 2nd axis (zero-indexed)
np.concatenate([a, a], axis=1)
```

```
array([[1, 2, 3, 1, 2, 3],
       [4, 5, 6, 4, 5, 6]])
```

```
b = np.array([[10], [11]])
np.hstack([a, b])
```

```
array([[ 1,  2,  3, 10],
       [ 4,  5,  6, 11]])
```

```
c = np.array([7, 8, 9])
np.vstack([a, c])
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Array operations

Basic operations

- Arithmetic operators: **elementwise** application.

```
a = np.array([0, 1, 2, 3])
b = np.array([2, 3, 2, 4])
print("-(0.5*a + 1) ** 2 = ", -(0.5*a + 1) ** 2)
print("a // 2 =", a // 2)   # floor division
print("a ** 2 = ", a ** 2)  # exponentiation
print("a % 2 = ", a % 2)    # modulus
print("a + b = ", a + b)
print("a * b = ", a * b)
```

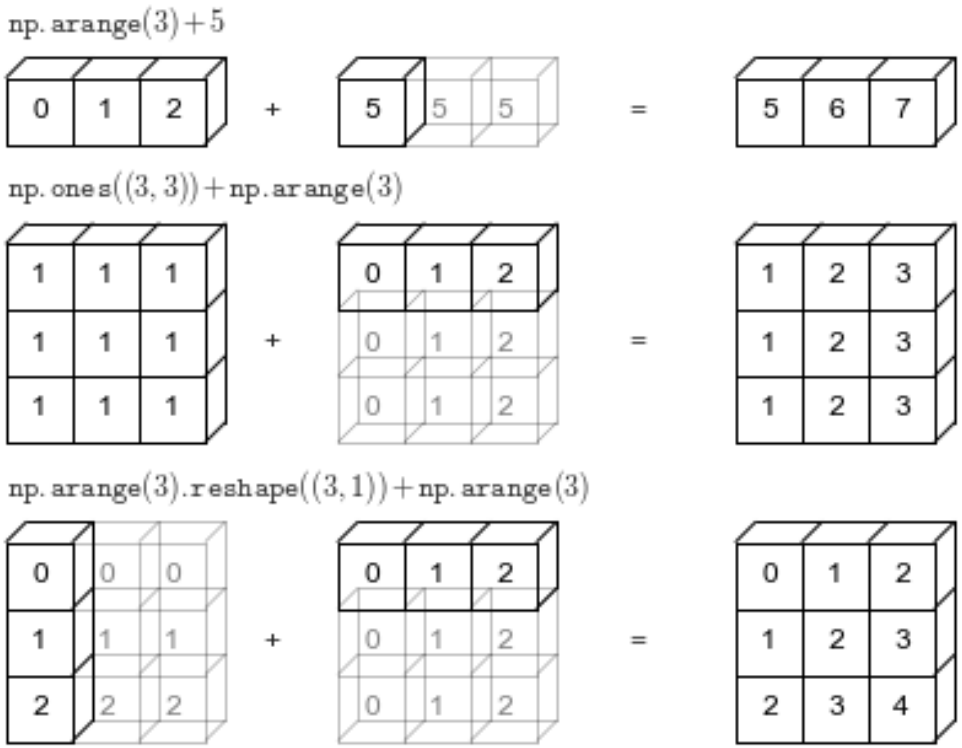
```
-(0.5*a + 1) ** 2 =  [-1.   -2.25 -4.   -6.25]
a // 2 = [0 0 1 1]
a ** 2 =  [0 1 4 9]
a % 2 =  [0 1 0 1]
a + b =  [2 4 4 7]
a * b =  [ 0  3  4 12]
```

- Also, we can use assignment operators, e.g., += and *=.

Array operations

Array broadcasting

- Broadcasting is simply a set of rules for applying basic operators (e.g., addition, subtraction, multiplication, etc.) on arrays of different shapes.
- When operating on two arrays, NumPy compares shapes. Two dimensions are compatible when:
  - They are of equal size
  - One of them is 1



Array operations

Vector operations

- Ways to multiply two or more vectors:
  - dot product → A scalar value
  - outer product → A matrix

```
# note: numpy automatically converts lists
u = [0, 1, 2]
v = [1, 2, 3]

print(np.dot(u, v))
```

```
8
```

```
print(np.outer(u, v))
```

```
array([[0, 0, 0],
       [1, 2, 3],
       [2, 4, 6]])
```

Array operations

Matrix operations

```
A = np.ones((3, 2))
print(A)
```

```
array([[1., 1.],
       [1., 1.],
       [1., 1.]])
```

```
A.T
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
B = np.ones((2, 3))
print(B)
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
np.dot(A, B)
```

```
array([[2., 2., 2.],
       [2., 2., 2.],
       [2., 2., 2.]])
```

```
np.dot(B, A)
```

```
array([[3., 3.],
       [3., 3.]])
```

```
np.dot(B.T, A.T)
```

```
array([[2., 2., 2.],
       [2., 2., 2.],
       [2., 2., 2.]])
```

Array operations

Aggregate operations

- Often when faced with a large amount of data, the first step is to compute summary statistics for the data.
- For example, mean, standard deviation, sum, product, median, minimum, maximum, quantiles, etc.

Array operations

```
a = np.array([[1,2,3], [4,5,6]])
print(a)
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

- Sum of an array or along an axis

```
a.sum()
```

```
21
```

```
a.sum(axis=0) # column sum
```

```
array([5, 7, 9])
```

- Cumulative sum of an array or along an axis

```
a.cumsum()
```

```
array([ 1,  3,  6, 10, 15, 21])
```

```
a.cumsum(axis=1) # cumulative row sum
```

```
array([[ 1,  3,  6],
       [ 4,  9, 15]])
```

- Min/Max of an array or along an axis

```
a.min()
```

```
1
```

```
a.max(axis=0)
```

```
array([4, 5, 6])
```

Linear algebra

- The linear algebra module **linalg** of NumPy offers various methods to apply linear algebra on any NumPy array, including:
  - rank, determinant, trace, etc. of an array.
  - eigen values of matrices
  - matrix and vector products (dot, inner, outer, etc. product), matrix exponentiation
  - solve linear or tensor equations and much more!

Linear algebra

linalg methods

```
import numpy as np
```

- `np.linalg.qr()` # Get qr factorization of a matrix
- `np.linalg.cholesky()` # Cholesky decomposition
- `np.linalg.inv(A)` # Inverse of a matrix
- `np.linalg.solve(A, b)` # Solves Ax = b
- `np.linalg.lstsq(A, b)` # Solves $\underset{x}{\operatorname{argmin}} \|Ax - b\|_2$
- `np.linalg.eigvals(A)` # Computes eigenvalues of a matrix
- `np.linalg.svd(A, full)` # Singular value decomposition
- `np.linalg.pinv(A)` # Computes pseudo-inverse of A

…

Further reading:
https://numpy.org/doc/stable/reference/routines.linalg.html

Fourier transform

- Fourier analysis is fundamentally a method for expressing a function as a sum of periodic components, and for recovering the function from those components.
- When both the function and its Fourier transform are replaced with discretized counterparts, it is called the discrete Fourier transform (DFT) and implemented by the fft module in NumPy.
- As the discrete Fourier transform separates its input into components that contribute at discrete frequencies, it has a great number of applications in digital signal processing, e.g., for filtering.

Fourier transform

fft (Fast Fourier Transform) methods

```
import numpy as np
```

- `np.fft.fft()` # 1-dimensional DFT
- `np.fft.fft2()` # 2-dimensional DFT
- `np.fft.fftn()` # N-dimensional DFT
- `np.fft.ifft()` # 1-dimensional inverse DFT
- `np.fft.rfft()` # Real DFT (1-dim)
- `np.fft.irfft()` # Inverse of rfft (1-dim))

…

Further reading:
https://numpy.org/doc/stable/reference/routines.fft.html

Random sampling

- NumPy offers the `random` module to produce *pseudo random* numbers.
- *Pseudo* Random number: It does NOT mean a different number every time. It means something that can not be predicted logically.
- *Pseudo* random vs true random:
  - If there is a program to generate random number, it can be predicted (as long as the initial conditions of the algorithm are provided), thus it is *pseudo* random.
  - To generate a truly random number on our computers, we need to get the random data from some outside source, e.g., mouse movements, data on network, etc.

Random sampling

random methods

```
import numpy as np
```

- `np.random.rand(d0, d1, ..., dn)` # Random from a uniform distribution over [0, 1) in a given shape
- `np.random.randn(d0, d1, ..., dn)` # Random from the standard normal distribution
- `np.random.randint(low, high, size)` # Random integers from "discrete uniform" distribution over [low, high)
- `np.random.choice(a, size, replace, p)` # Sample from 1D array a
- `np.random.shuffle(a)` # Permutation (in-place)
- `np.random.permutation(a)` # Permutation (new array)

…

```
import numpy as np
np.random.rand(3,2)
```

```
array([[0.11558665, 0.75991703],
       [0.75468789, 0.8191761 ],
       [0.04962621, 0.15843178]])
```

Random sampling

Distributions in random

- Sample from different probability distributions (parameterized).

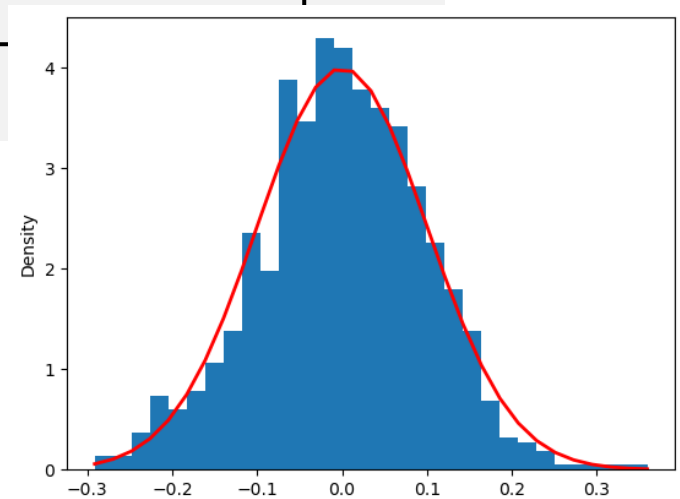
```
import numpy as np
```

- `np.random.normal()`
- `np.random.beta()`
- `np.random.binomial()`
- `np.random.chisquare()`
- `np.random.exponential()`
- `np.random.dirichlet()`
- `np.random.gamma()`
- `np.random.laplace()`
- `np.random.lognormal()`
- `np.random.pareto()`
- `np.random.poisson()`
- `np.random.power()`

…

Further reading:
https://numpy.org/doc/stable/reference/random/legacy.html

Random sampling

Example: Sampling from a normal distribution $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$,

```
import numpy as np

# Draw samples from the normal distribution
mu, sigma = 0, 0.1 # mean and standard deviation
s = np.random.normal(mu, sigma, 1000)

# Plot histogram of the samples and the probability density function
import matplotlib.pyplot as plt
count, bins, ignored = plt.hist(s, 30, density=True)

plt.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) *
                np.exp( - (bins - mu)**2 / (2 * sigma**2) ),
         linewidth=2, color='r')
plt.ylabel('Density')
plt.show()
```